

Projeto NotaDez

Integrantes:

João Paulo Ferreira – 25000684

Leonardo Miranda Nunciaroni – 25002726

Miguel Fernandes Costacurta – 25003110

Pedro Henrique Contardi Soler – 25005592

1. Apresentação do Descritivo do Projeto

O NotaDez é uma aplicação web voltada para docentes do ensino superior, desenvolvida para facilitar o gerenciamento de notas e avaliações. O sistema oferece uma alternativa prática e segura às planilhas eletrônicas e aos sistemas institucionais complexos, centralizando o controle de dados acadêmicos e garantindo sua integridade por meio de logs de auditoria.

A aplicação permite o cadastro de instituições, disciplinas, turmas e alunos, além da importação e exportação de dados via CSV, lançamento de notas com cálculo automático de médias e acesso remoto via web. Foi desenvolvida com Node.js e TypeScript no backend, HTML5 e CSS3 no frontend, e MySQL como banco de dados, seguindo boas práticas de engenharia de software e metodologias ágeis.

Objetivos Principais

- Facilitar o gerenciamento acadêmico dos professores por meio de uma ferramenta intuitiva e centralizada.
- Assegurar a integridade e rastreabilidade dos dados com histórico completo e logs de auditoria.
- Automatizar tarefas repetitivas, como cálculos e importações de alunos.
- Aplicar conceitos técnicos de engenharia de software, desenvolvimento e versionamento de código.

Objetivos Específicos

- Criar um sistema CRUD completo para todas as entidades acadêmicas.
- Implementar autenticação segura e recuperação de senha.
- Permitir importação e exportação de dados em CSV.
- Disponibilizar interface de lançamento de notas com validações e cálculo automático de médias.
- Desenvolver um sistema de auditoria via triggers no banco de dados.
- Garantir documentação completa do projeto com diagramas, README e comentários no código.

O NotaDez busca oferecer uma solução moderna e funcional que atenda às necessidades reais dos docentes, ao mesmo tempo em que demonstra a aplicação prática dos conhecimentos técnicos adquiridos em engenharia de software.

1.1. Liste os Requisitos Funcionais

1. Autenticação

RF001 – O sistema deve permitir que o docente crie uma conta informando nome completo, e-mail, telefone e senha.

RF002 – O sistema deve validar as credenciais de acesso do docente no momento do login.

RF003 – O sistema deve permitir a recuperação de senha por meio de link enviado ao e-mail cadastrado.

RF004 – O sistema não deve permitir acesso anônimo ou modo visitante.

RF005 – O sistema deve exibir a tela de autenticação como página inicial.

2. Gerenciamento de Instituições, Disciplinas e Turmas

RF006 – O sistema deve permitir que o docente cadastre uma ou mais instituições onde atua.

RF007 – O sistema deve permitir que o docente edite e exclua instituições previamente cadastradas.

RF008 – O sistema deve impedir a exclusão de uma instituição que possua disciplinas vinculadas.

RF009 – O sistema deve permitir que o docente cadastre disciplinas informando nome, código, sigla e período.

RF010 – O sistema deve permitir que o docente associe disciplinas a uma instituição cadastrada.

RF011 – O sistema deve permitir a criação, edição e exclusão de turmas associadas a uma disciplina.

RF012 – O sistema deve solicitar confirmação por e-mail antes da exclusão de turmas que possuam notas lançadas.

RF013 – O sistema deve impedir a exclusão de disciplinas que possuam turmas associadas.

3. Cadastro e Importação de Alunos

RF014 – O sistema deve permitir o cadastro manual de alunos informando matrícula e nome completo.

RF015 – O sistema deve permitir a importação de alunos a partir de arquivos no formato CSV.

RF016 – O sistema deve permitir a importação de alunos a partir de arquivos no formato JSON.

RF017 – O sistema deve considerar apenas as duas primeiras colunas do arquivo CSV, sendo a primeira o identificador do aluno e a segunda o nome completo.

RF018 – O sistema deve verificar duplicidades de identificadores durante a importação e manter os dados originais do sistema como fonte da verdade.

RF019 – O sistema deve permitir a exclusão de alunos individualmente ou por seleção múltipla dentro de uma turma.

4. Componentes de Nota

RF020 – O sistema deve permitir que o docente crie componentes de nota, informando nome, sigla e descrição.

RF021 – O sistema deve aceitar valores numéricos entre 0.00 e 10.00, com duas casas decimais de precisão, para cada componente de nota.

RF022 – O sistema deve apresentar os componentes de nota em uma tabela organizada por aluno e componente.

5. Lançamento de Notas

RF023 – O sistema deve permitir o lançamento de notas individuais por componente.

RF024 – O sistema deve oferecer um modo de edição completa, permitindo a modificação de notas de múltiplos componentes simultaneamente.

RF025 – O sistema não deve permitir valores fora do intervalo de 0.00 a 10.00.

RF026 – O sistema deve impedir a edição acidental de notas quando o modo de edição estiver desativado.

6. Painel de Auditoria

RF027 – O sistema deve registrar toda alteração de nota com data, hora, usuário e valores antigos e novos.

RF028 – O sistema deve exibir as mensagens de auditoria em tempo real no painel lateral da tela de notas.

RF029 – O sistema deve armazenar permanentemente os registros de auditoria no banco de dados.

RF030 – O sistema deve exibir os registros de auditoria em ordem decrescente de data e hora.

RF031 – O sistema deve permitir que o docente oculte o painel de auditoria, sem desativar sua funcionalidade.

7. Cálculo e Ajuste de Notas Finais

RF032 – O sistema deve permitir que o docente cadastre uma fórmula matemática para o cálculo da nota final da disciplina.

RF033 – O sistema deve validar se todos os componentes cadastrados estão presentes na fórmula antes de salvá-la.

RF034 – O sistema deve calcular automaticamente a nota final de cada aluno com base na fórmula informada.

RF035 – O sistema não deve permitir a edição manual da coluna de notas finais automáticas, geradas a partir da fórmula cadastrada.

RF036 – O sistema pode permitir a criação de uma coluna de notas finais ajustadas quando o docente selecionar essa opção.

RF037 – O sistema deve arredondar automaticamente as notas finais ajustadas para múltiplos de 0.5, conforme regras definidas.

RF038 – O sistema deve permitir que o docente altere manualmente as notas finais ajustadas respeitando os intervalos de 0.5 em 0.5.

8. Exportação de Dados

RF039 – O sistema deve permitir a exportação de notas e informações de alunos em formato CSV.

RF040 – O sistema deve permitir a exportação de notas e informações de alunos em formato JSON.

RF041 – O sistema deve impedir a exportação se houver notas em branco.

RF042 – O sistema deve gerar o nome do arquivo exportado seguindo o padrão YYYY-MM-DD_HHmmss_Turma_Sigla.csv ou json.

RF043 – O sistema não deve armazenar os arquivos exportados no servidor após o download.

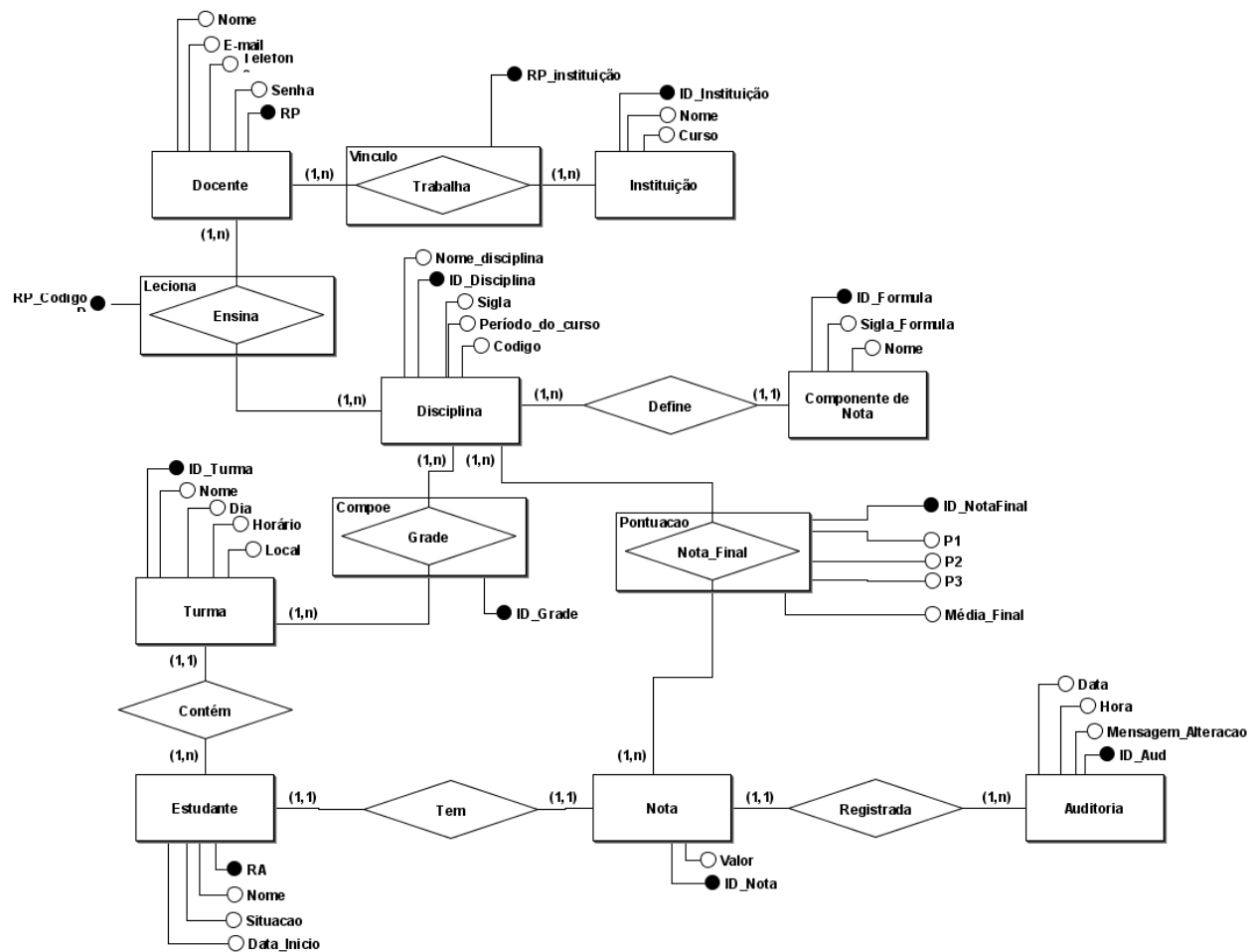
9. Interface e Hospedagem

RF044 – O sistema deve ser 100% web, acessível nos navegadores Chrome, Firefox e Edge.

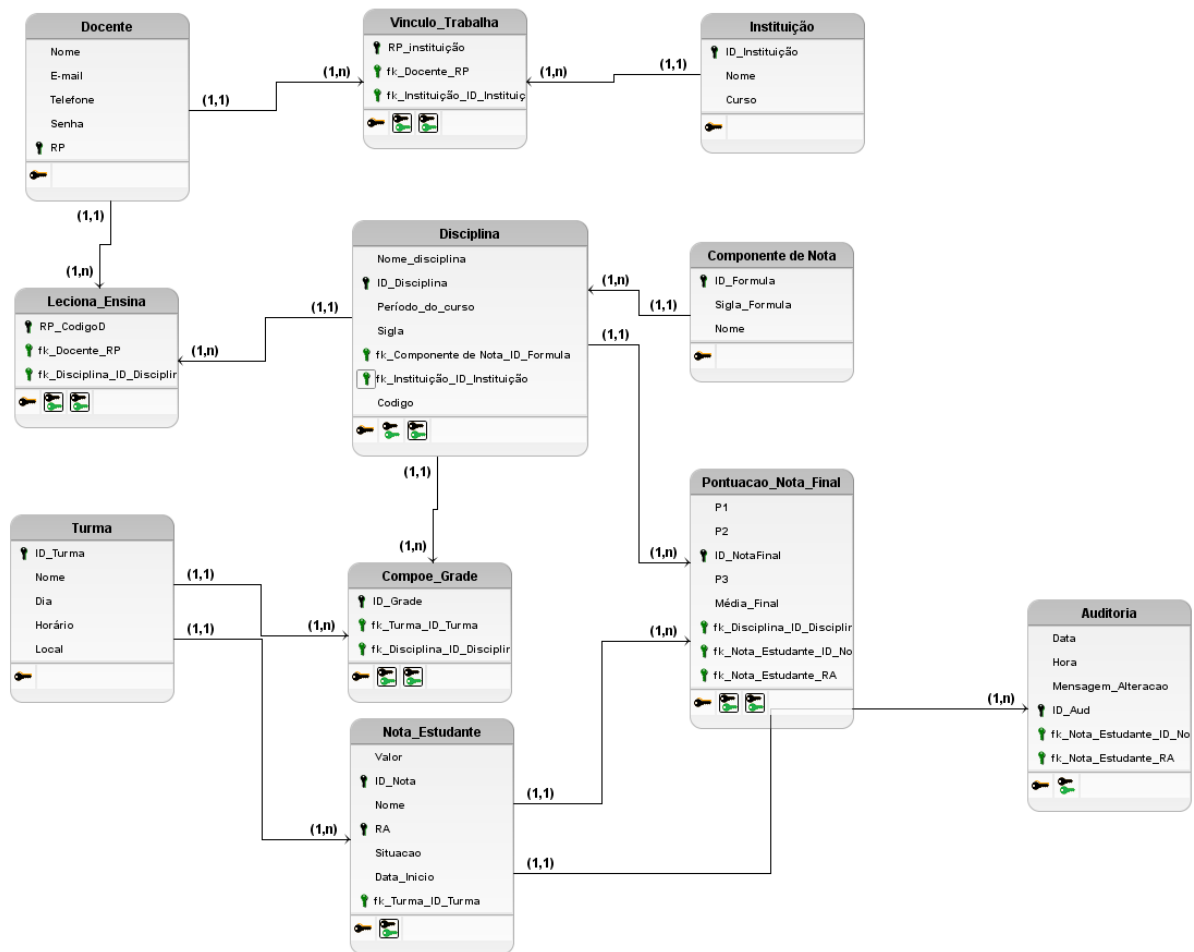
RF045 – O sistema deve ser responsivo e adaptar-se automaticamente a diferentes tamanhos de tela.

RF046 – O sistema deve permitir hospedagem em nuvem ou servidor dedicado, garantindo acesso remoto.

2. Modelagem do Sistema – Modelo Conceitual (MER)



3. Fazer o Modelo Lógico (usar softwares de modelagem)



4. Projeto de Banco de Dados – Modelo Físico (Criar no SQL developer – Colocar a estrutura da tabela criada.

```
-- =====
-- TABELA DOCENTE
-- =====

CREATE TABLE Docente (
    RP NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Nome VARCHAR2(255),
    Email VARCHAR2(255) UNIQUE,
    Telefone VARCHAR2(20),
    Senha VARCHAR2(255)
);

-- =====
-- TABELA INSTITUICAO
-- =====

CREATE TABLE Instituicao (
    ID_Instituicao NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Nome VARCHAR2(255),
    Curso VARCHAR2(255) NOT NULL
);

-- =====
-- TABELA VINCULO
-- =====

CREATE TABLE Vinculo (
    RP_Instituicao NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    RP NUMBER NOT NULL,
    ID_Instituicao NUMBER NOT NULL,
    CONSTRAINT fk_trabalha_docente FOREIGN KEY (RP)
        REFERENCES Docente (RP),
    CONSTRAINT fk_trabalha_instituicao FOREIGN KEY (ID_Instituicao)
        REFERENCES Instituicao (ID_Instituicao)
);

-- =====
-- TABELA COMPONENTE_DE_NOTA
-- =====

CREATE TABLE Componente_de_Nota (
    ID_Formula NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Nome VARCHAR2(100),
    Sigla_Formula VARCHAR2(20)
);

-- =====
```

-- TABELA DISCIPLINA

-- =====

```
CREATE TABLE Disciplina (  
    ID_Disciplina NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    Nome_disciplina VARCHAR2(255),  
    Sigla VARCHAR2(10),  
    Codigo VARCHAR2(20),  
    Periodo_do_curso NUMBER(2),  
    ID_Formula NUMBER,  
    ID_Instituicao NUMBER,  
    CONSTRAINT fk_disciplina_componente FOREIGN KEY (ID_Formula)  
        REFERENCES Componente_de_Nota (ID_Formula),  
    CONSTRAINT fk_disciplina_instituicao FOREIGN KEY (ID_Instituicao)  
        REFERENCES Instituicao (ID_Instituicao)  
);
```

-- =====

-- TABELA ENSINA (N:M entre Docente e Disciplina)

-- =====

```
CREATE TABLE Ensina (  
    RP_CodigoD NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    RP NUMBER NOT NULL,  
    ID_Disciplina NUMBER NOT NULL,  
    CONSTRAINT fk_ensina_docente FOREIGN KEY (RP)  
        REFERENCES Docente (RP),  
    CONSTRAINT fk_ensina_disciplina FOREIGN KEY (ID_Disciplina)  
        REFERENCES Disciplina (ID_Disciplina)  
);
```

-- =====

-- TABELA TURMA

-- =====

```
CREATE TABLE Turma (  
    ID_Turma NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    Nome VARCHAR2(100),  
    Dia VARCHAR2(50),  
    Horario VARCHAR2(50),  
    Local_ VARCHAR2(100)  
);
```

-- =====

-- TABELA GRADE (N:M entre Disciplina e Turma)

-- =====

```
CREATE TABLE Grade (  
    ID_Grade NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    ID_Disciplina NUMBER NOT NULL,  
    ID_Turma NUMBER NOT NULL,
```

```
CONSTRAINT fk_grade_disciplina FOREIGN KEY (ID_Disciplina)
REFERENCES Disciplina (ID_Disciplina),
CONSTRAINT fk_grade_turma FOREIGN KEY (ID_Turma)
REFERENCES Turma (ID_Turma)
);
```

```
-- =====
-- TABELA ESTUDANTE
-- =====
```

```
CREATE TABLE Estudante (
  RA VARCHAR2(20) PRIMARY KEY,
  Nome VARCHAR2(255),
  Situacao VARCHAR2(50),
  Data_Inicio DATE,
  ID_Turma NUMBER NOT NULL,
  CONSTRAINT fk_estudante_turma FOREIGN KEY (ID_Turma)
  REFERENCES Turma (ID_Turma)
);
```

```
-- =====
-- TABELA NOTA
-- =====
```

```
CREATE TABLE Nota (
  ID_Nota NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Valor NUMBER(5,2),
  RA VARCHAR2(20) NOT NULL,
  CONSTRAINT fk_nota_estudante FOREIGN KEY (RA)
  REFERENCES Estudante (RA)
);
```

```
-- =====
-- TABELA NOTA_FINAL
-- =====
```

```
CREATE TABLE Nota_Final (
  ID_NotaFinal NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  P1 NUMBER(5,2),
  P2 NUMBER(5,2),
  P3 NUMBER(5,2),
  Media_Final NUMBER(5,2),
  ID_Grade NUMBER NOT NULL,
  ID_Nota NUMBER NOT NULL,
  CONSTRAINT fk_nota_final_grade FOREIGN KEY (ID_Grade)
  REFERENCES Grade (ID_Grade),
  CONSTRAINT fk_nota_final_nota FOREIGN KEY (ID_Nota)
  REFERENCES Nota (ID_Nota)
);
```

```
-- =====  
-- TABELA AUDITORIA  
-- =====  
CREATE TABLE Auditoria (  
    ID_Aud NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    Data_Hora TIMESTAMP,  
    Mensagem_Alteracao CLOB,  
    ID_Nota NUMBER NOT NULL,  
    CONSTRAINT fk_auditoria_nota FOREIGN KEY (ID_Nota)  
        REFERENCES Nota (ID_Nota)  
);
```

5. Considerações Finais

O presente trabalho consistiu na elaboração das etapas de modelagem de banco de dados para o sistema NotaDez, conforme as diretrizes da disciplina Estudos de Banco de Dados I. Partindo da especificação detalhada do escopo e dos requisitos funcionais do Projeto Integrador 2, o grupo desenvolveu o Modelo Conceitual (MER), o Modelo Lógico e o Modelo Físico, estabelecendo a estrutura SQL necessária para um banco de dados relacional.

O principal resultado obtido até o momento é um sistema de banco de dados normalizado, que serve como fundação segura para a implementação do backend em Node.js e TypeScript no projeto.

Os resultados concretos incluem a modelagem das seguintes estruturas:

- **Entidades de Autenticação:** Modelagem da entidade de usuário para suportar o cadastro e o login do docente (Nome, e-mail, celular, senha), um ponto de partida crucial dado que não há acesso anônimo.
- **Hierarquia de Dados:** Estrutura clara para o gerenciamento de Instituições, Disciplinas e Turmas, respeitando as relações de dependência necessárias para a exclusão sequencial.
- **Gerenciamento de Notas Complexo:** Criação de estruturas para Componentes de Nota, as próprias notas, o armazenamento da Expressão Matemática da nota final e as colunas de Nota Final Automática e Nota Final Ajustada (com arredondamento de 0.5 em 0.5).
- **Rastreabilidade e Segurança:** Design da entidade de Auditoria (LOG) para registrar e persistir cada alteração ou lançamento de nota, garantindo a rastreabilidade e a confiança do docente nas operações.

Durante o processo de modelagem, foram identificados e enfrentados diversos desafios técnicos e de regras de negócio de alta criticidade:

- **Desafio da Autenticação, Login e Verificação por E-mail:** A modelagem da autenticação é sensível, mas a implementação dos mecanismos de segurança para o login e a obrigatoriedade do recurso de recuperação de senha por e-mail adicionam uma camada de complexidade na gestão de tokens e na integração com o servidor de envio de mensagens.
- **Desafio da Persistência de Dados e Integração (MySQL):** A escolha por um dos bancos relacionais obrigatórios apresenta o desafio técnico de garantir a integração eficiente com o backend em Node.js/TypeScript. Especificamente com o MySQL, a otimização do sistema para suportar a alta volume de dados da auditoria e o desempenho das consultas complexas para o cálculo de notas são pontos de constante atenção.

- **Desafio da Regra de Negócio de Exclusão e Confirmação:** A modelagem da exclusão de Turmas exige um controle rigoroso de chaves estrangeiras. No entanto, o principal desafio se concentra na implementação da confirmação de exclusão por link no e-mail, que deve garantir que o docente não exclua dados críticos acidentalmente.
- **Desafio da Auditoria:** Modelar o sistema de auditoria (LOG) para salvar todas as modificações de nota em tempo real, garantindo a persistência dos detalhes no banco de dados e a ordenação por data/hora decrescente sem comprometer o desempenho da aplicação.